How does violating homoscedasticity affect inference in linear regression?

Yanqi Huang, MS Student[1]

Abstract

Ordinary least squares (OLS) regression relies on the assumption of constant error variance. This simulation study shows that while OLS estimates remain unbiased under heteroscedasticity, model-based standard errors underestimate true variability, leading to confidence intervals with poor coverage. These results highlight the need to account for heteroscedasticity to ensure valid inference.

Introduction

Ordinary least squares (OLS) regression is widely used to explore relationships between explanatory factors and an outcome variable. For example, in biomedical research, it is often applied to assess associations between risk factors and health outcomes. The key assumptions of OLS include linearity, normality of errors, and homoscedasticity.The purpose of this study is to evaluate how varying levels of heteroscedasticity affect inference in OLS regression using simulation.

Motivating Problem

In practice, the assumption of homoscedasticity is frequently overlooked when fitting linear regression models. However, violations of this assumption can have a greater impact on the validity of regression results than violations of normality [1]. This simulation study examined the impact of mild, moderate, and strong levels of heteroscedasticity on bias, model based standard error(SE), empirical standard error, their relative error and confidence interval (CI) coverage in ordinary OLS regression.

Simulation Set-Up

To evaluate how key performance metrics vary under increasing heteroscedasticity, the simulation was designed as follows:

1. The predictor x_i is generated from a uniform distribution $x_i \sim \text{Uniform}(1, 6)$ with a sample size of $n = 100$ [2].
2. The error variance is defined as a function of x, with parameter c controlling the degree of heteroscedasticity:

$$\varepsilon_i \sim \mathcal{N}\left(0,\ \frac{\exp\{c(x_i - \mu_x)\}}{\mathbb{E}[\exp\{c(x_i - \mu_x)\}]}\right)$$

   - $c = 0$ (homoscedastic baseline): The variance simplifies to 1 for all x_i, giving

$$\varepsilon_i \sim \mathcal{N}(0, 1)$$

   - $c > 0$: three levels of heteroscedasticity were examined: c=0.3 (mild), where variance increases only slightly with x_i; c=1 (moderate), where variance grows more noticeably across the range of x_i; c=6 (strong), where variance differs substantially with x_i.
3. The response variable is defined as

$$y_i = 14 + 0.4x_i + \varepsilon_i$$

   with the true slope $\beta_1 = 0.4$ [2].
4. Each scenario is replicated 1000 times, consistent with the average number of iterations used in previous studies [2].

Residual plots against fitted values in Figure 1 show that increasing c produces greater spread of residuals with $\hat{y}$, reflecting progressively stronger heteroscedasticity.

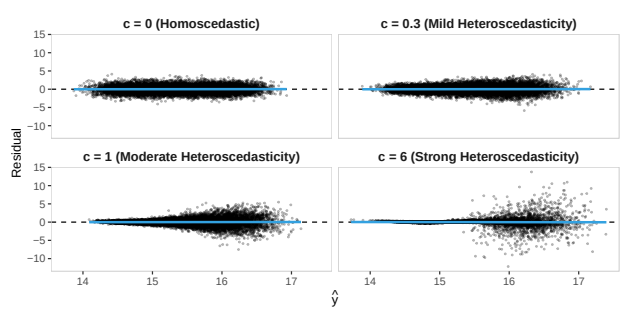

Figure 1: Residuals vs. fitted values for OLS regression under Homoscedasticity(c=0) and increasing levels of heteroscedasticity(c=0.3,1,6)

To isolate heteroscedasticity itself, the variance function is normalized so that the average error variance $\mathbb{E}[\text{Var}(\varepsilon_i)]$ is fixed at 1 (Table 1, left panel). This adjustment keeps the noise level constant and allows fair

[1]Department of Biostatistics and Informatics, Colorado School of Public Health, University of Colorado Anschutz Medical Campus, Aurora, CO

comparison between homoscedastic and heteroscedastic settings. Without adjustment, both the average error variance and the variance of Y rise together, confounding heteroscedasticity with overall noise (Table 1, right panel)

Table 1: Variance of Y and expected error variance across heteroscedasticity levels (c)

	Normalized		Unnormalized	
c	$\mathrm{Var}(Y)$	$\mathbb{E}[\mathrm{Var}(\varepsilon \mid X)]$	$\mathrm{Var}(Y)$	$\mathbb{E}[\mathrm{Var}(\varepsilon \mid X)]$
0	1	1	1	1
0.3	1	1	1	1
1	1	1	3	2
6	1	1	110319	109623

Results

Figure 2 summarizes the effect of increasing heteroscedasticity(c) on the estimation of $\hat{\beta}_1$, with 95% Monte Carlo confidence intervals. Bias stayed near zero across all settings, confirming that OLS provided unbiased slope estimates. In contrast, variability measures diverged: the empirical SE (true variability across simulations) rose from 0.069 at c=0 to 0.113 at c=6, while the model based SE (average OLS reported SE) fell slightly from 0.069 to 0.065. As a result, the model based SE increasingly underestimated true variability, with relative error reaching –42.1% at c=6. Correspondingly, CI coverage declined from 94.6% under homoscedasticity to 73.4% under strong heteroscedasticity.

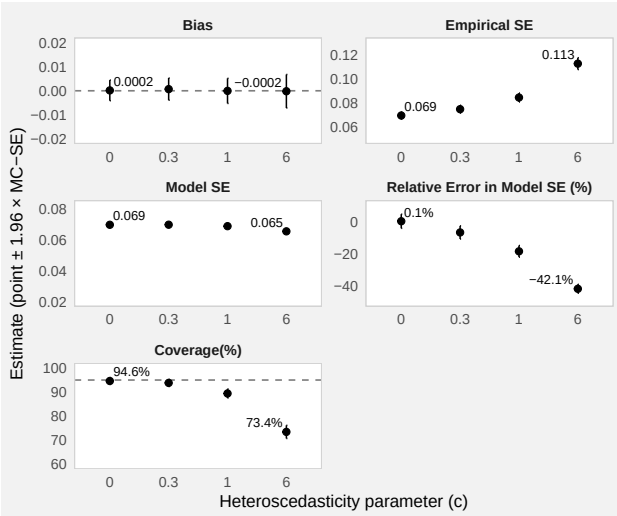

Figure 2: Metrics with 95% Monte Carlo Confidence Intervals across Levels of Heteroscedasticity

While Figure 2 illustrates the overall trends, Table 2 provides the corresponding numerical results with Monte Carlo standard errors, showing the precision of the simulation estimates. For example, bias was consistently negligible, with all estimates within ±0.004. at c=6 the empirical standard error was 0.113 (±0.003) compared to a model based standard error of 0.065 (±0.001) and coverage estimates within ±1.4.

Table 2: Simulation results for OLS regression under heteroscedasticity with Monte Carlo standard errors (in parentheses)

c	Bias	Model SE	Empirical SE	Relative error in model SE (%)	CI Coverage (%)
0	0.0002 (±0.002)	0.069 (±0.000)	0.069 (±0.002)	0.1 (±2.3)	94.6 (±0.7)
0.3	0.0007 (±0.002)	0.070 (±0.000)	0.075 (±0.002)	-6.9 (±2.1)	93.8 (±0.8)
1	-0.0000 (±0.003)	0.069 (±0.000)	0.084 (±0.002)	-18.7 (±1.8)	89.4 (±1.0)
6	-0.0002 (±0.004)	0.065 (±0.001)	0.113 (±0.003)	-42.1 (±1.4)	73.4 (±1.4)

Conclusion

These simulations show that OLS slope estimates remain unbiased under heteroscedasticity, but inference based on model based standard errors is unreliable. As heteroscedasticity increased, empirical variability rose while model based SEs declined slightly, resulting in confidence interval coverage falling up to 20 percentage points below nominal 95%. Thus, OLS-based inference gives overly optimistic precision, and robust alternatives should be preferred such as heteroscedasticity consistent SEs or alternative models.

This study is limited to one covariate distribution, a fixed sample size, and a single form of heteroscedasticity. Future work could examine other covariate distributions, larger samples, and alternative variance structures. Overall, careful attention to homoscedasticity assumption is essential, as violations can lead to misleadingly narrow intervals and incorrect conclusions.

References

Code Appendix

```
knitr::opts_chunk$set(echo = TRUE)
#bibliography: kaizer_simulation_sample1.bib
#csl: alex_vancouver.csl
## ================================
## 0. Load needed packages and data
## ================================
# Helper function: install if not already available
load_or_install <- function(pkg){
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
  library(pkg, character.only = TRUE)
}

# List of needed packages
packages <- c("dplyr", "ggplot2",  "kableExtra", "webshot2","magick","knitr","tidyverse","tidyr","ggh4x"
# Load or install each
lapply(packages, load_or_install)


# Load simulation results
path <- "~/Library/CloudStorage/Dropbox/School/CU/fall 2025/BIOS 6618 adv biostatistical method/midterm
fits_file <- '/data/2025-10-22_optimized-seed-best-param-centered-log-linear-fits_ols-heteroscedasticity
simulated_data_file <- '/data/2025-10-22_optimized-seed-best-param-centered-log-linear-datasets_ols-hete
fits <- readRDS(paste0(path, fits_file))
simulated_data <- readRDS(paste0(path, simulated_data_file))
# Extract parameter settings from filename for labeling outputs
param_data <- sub(".*_(n[0-9].*).rds$", "\\1", basename(fits_file))
message("Analyzing results for parameter settings: ", param_data)
true_beta <- unique(fits$beta1_true)
c_params <- c(0,0.3,1,6)
# ISO 8601 date (date only):
iso_date <- format(Sys.Date(), "%Y-%m-%d")
save_results_path <- paste0(path,"/results/",iso_date)

if (!dir.exists(save_results_path)) {
  dir.create(save_results_path, recursive = TRUE)
}

## ================================
## 1. Summarize results
## ================================
# Summarize datasets to check variances
average_error_variance <- simulated_data %>%
  group_by(c_param) %>%
  summarise(
    var_y = round(var(y),0),                        # variance of Y
    mean_eps_var = round(mean(eps_var),0),          # average
    var_y_unnorm = round(var(y_unnorm),0),
```

```r
    mean_eps_var_unnorm = round(mean(eps_var_unnorm), 0),
    .groups = "drop"
  )
average_error_variance <- average_error_variance %>%
  mutate(c_param = ifelse(c_param %% 1 == 0,
                          as.character(as.integer(c_param)),
                          as.character(c_param)))
# Summarize simulation results with MC-SEs
fits_summ <- fits %>%
  mutate(
    covered   = as.integer(true_beta >= ci_lower & true_beta <= ci_upper),
    rejected  = as.integer(p_value <= 0.05),
    est_err   = beta1_hat - true_beta
  ) %>%
  group_by(c_param) %>%
  summarise(
    R = n(),  # number of replications per c

    # Bias and its MC-SE
    bias         = mean(est_err, na.rm = TRUE),
    mcse_bias    = sd(est_err, na.rm = TRUE) / sqrt(R),

    # SD of beta1_hat across reps, with delta-method MC-SE: sd / sqrt(2*(R-1))
    sd_beta1_hat = sd(beta1_hat, na.rm = TRUE),
    mcse_sd_beta1 = sd_beta1_hat / sqrt(2 * pmax(R - 1, 1)),

    # Mean (reported) SE and its MC-SE
    se_beta1_hat = mean(se, na.rm = TRUE),
    mcse_se_mean = sd(se, na.rm = TRUE) / sqrt(R),

    # Relative error of model SE vs empirical SD
    rel_err_se    = (se_beta1_hat - sd_beta1_hat) / sd_beta1_hat,

    # --- MCSE for rel_err_se via delta method ---
    # Notation: M = mean(se), S = sd(beta1_hat)
    # g(M, S) = M/S - 1; grad g = (1/S, -M/S^2)
    varM          = var(se, na.rm = TRUE) / R,
    varS          = (sd_beta1_hat^2) / (2 * pmax(R - 1, 1)),
    mcse_rel_err  = sqrt( (1 / sd_beta1_hat)^2 * varM +
                          (se_beta1_hat^2 / sd_beta1_hat^4) * varS ),
    # Coverage and its MC-SE
    ci_coverage  = mean(covered, na.rm = TRUE),
    mcse_cov     = sqrt(ci_coverage * (1 - ci_coverage) / R),



    .groups = "drop"
  )

## ================================
## 2. Create figure with metrics and 95% MC-SE error bars
## ================================
# Create table with formatted metrics including MC-SEs
## 1) Find rows using numeric values from fits_summ
```

```r
row_min_relerr <- which.min(fits_summ$rel_err_se)      # rel_err_se is in proportion (not %)
row_min_cov    <- which.min(fits_summ$ci_coverage)     # proportion (0-1)
row_max_empse  <- which.max(fits_summ$sd_beta1_hat)

## 2) Build the display table (your code)
fits_table <- fits_summ %>%
  mutate(
    c_param       = ifelse(c_param %% 1 == 0,
                           as.character(as.integer(c_param)),
                           as.character(c_param)),
    bias          = sprintf("\\makecell{%.4f \\\\ (±%.3f)}", bias, mcse_bias),
    se_beta1_hat  = sprintf("\\makecell{%.3f \\\\ (±%.3f)}", se_beta1_hat, mcse_se_mean),
    sd_beta1_hat  = sprintf("\\makecell{%.3f \\\\ (±%.3f)}", sd_beta1_hat, mcse_sd_beta1),
    rel_err_se    = sprintf("\\makecell{%.1f \\\\ (±%.1f)}", rel_err_se*100, mcse_rel_err*100),
    ci_coverage   = sprintf("\\makecell{%.1f \\\\ (±%.1f)}", ci_coverage*100, mcse_cov*100)
  ) %>%
  select(c_param, bias, se_beta1_hat, sd_beta1_hat, rel_err_se, ci_coverage)
my_labels <- c(
  "0" = "c = 0 (Homoscedastic)",
  "0.3" = "c = 0.3 (Mild Heteroscedasticity)",
  "1" = "c = 1 (Moderate Heteroscedasticity)",
  "6" = "c = 6 (Strong Heteroscedasticity)"
)

ggplot(simulated_data %>% slice_sample(n = 48000),
  aes(y_hat, resid)) +
  geom_point(alpha = 0.3, size = 0.4) +
  geom_hline(yintercept = 0, linetype = 2) +
  geom_smooth(method = "lm", se = FALSE, formula = y ~ x, color = "#33A1E0") +
  facet_wrap(
    ~ c_param,
    ncol = 2,
    labeller = labeller(c_param = as_labeller(my_labels)),
    scales = "fixed"
  ) +
  labs(x = expression(hat(y)), y = "Residual") +
  theme_bw(base_size = 12) +
  theme(
    panel.grid = element_blank(),            # no grid lines
    strip.background = element_blank(),      # no gray background for facet labels
    strip.text = element_text(size=12,face = "bold"),
    panel.border = element_rect(color = "lightgray", fill = NA, linewidth = 0.5) # light gray box
  )
average_error_variance |>
  kbl(
    booktabs = TRUE,
    escape   = FALSE,
    digits   = c(0, 3, 3, 3, 3),
    col.names = c(
      "$c$",
      "$\\mathrm{Var}(Y)$",
      "$\\mathbb{E}[\\mathrm{Var}(\\varepsilon\\mid X)]$",
      "$\\mathrm{Var}(Y)$",
      "$\\mathbb{E}[\\mathrm{Var}(\\varepsilon\\mid X)]$"
```

```r
    ),
    caption = "Variance of $Y$ and expected error variance across heteroscedasticity levels ($c$)"
  ) |>
  add_header_above(c(" " = 1, "Normalized" = 2, "Unnormalized" = 2)) |>
  kable_styling(
    latex_options = c("HOLD_position","scale_down"),
    full_width    = FALSE,
    position      = "center",
    font_size     = 7
  )


metrics_long <- bind_rows(
  fits_summ %>% transmute(c_param, metric = "Bias",                  est = bias,             mcse = mcse_
  fits_summ %>% transmute(c_param, metric = "Empirical SE",          est = sd_beta1_hat,     mcse = mcse_
  fits_summ %>% transmute(c_param, metric = "Model SE",              est = se_beta1_hat,     mcse = mcse_
  fits_summ %>% transmute(c_param, metric = "Relative Error in Model SE (%)",est = round(rel_err_se*100,
  fits_summ %>% transmute(c_param, metric = "Coverage(%)",           est = ci_coverage*100,  mcse = mcse_
) %>%
  mutate(
    lo = est - 1.96 * mcse,
    hi = est + 1.96 * mcse,
    ref = case_when(
      metric == "Bias"        ~ 0,
      metric == "Coverage(%)" ~ 95,
      TRUE ~ NA_real_
    ),
    metric  = factor(metric, levels = c("Bias","Empirical SE","Model SE","Relative Error in Model SE (%)
    c_factor = factor(c_param, levels = c_params)
  )

# Make sure c_param is treated as factor with correct order
metrics_long$c_param <- factor(
  metrics_long$c_param,
  levels = sort(unique(metrics_long$c_param))
)

# Labels only for c=0 and c=6
label_data <- metrics_long %>%
  filter(c_param %in% c("0", "6")) %>%
  mutate(
    metric_chr = as.character(metric),
    label_val = case_when(
      grepl("%", metric_chr) ~ sprintf("%.1f%%", est),    # 1 decimal + %
      metric_chr == "Bias"   ~ sprintf("%.4f", est),      # 4 decimals for Bias
      TRUE                   ~ sprintf("%.3f", est)       # otherwise 3 decimals
    ),
    hjust_val = ifelse(c_param == "0", -0.1, 1.1),
    vjust_val = -0.5
  )


p <- ggplot(metrics_long, aes(x = c_param, y = est)) +
  geom_hline(aes(yintercept = ref), linetype = "dashed", na.rm = TRUE, color = "grey50") +
```

```r
  geom_errorbar(aes(ymin = lo, ymax = hi), width = 0, linewidth = 0.5, color = "black") +
  geom_point(size = 2, color = "black") +
  geom_text(
    data = label_data,
    aes(label = label_val, hjust = hjust_val, vjust = vjust_val),
    size = 3, color = "black"
  ) +
  ggh4x::facet_wrap2(~ metric, scales = "free_y", ncol = 2, axes = "all") +
  # facet-specific y-limit for Model SE
  ggh4x::facetted_pos_scales(
    y = list(
      metric == "Model SE" ~ scale_y_continuous(limits = c(0.02, 0.082)),
      metric == "Bias" ~ scale_y_continuous(limits = c(-0.02, 0.02)),
      metric == "Coverage(%)" ~ scale_y_continuous(limits = c(60, 100)),
      metric == "Empirical SE" ~ scale_y_continuous(limits = c(0.05, 0.13)),
      metric == "Relative Error in Model SE (%)" ~ scale_y_continuous(limits = c(10, -50))
    )
  ) +
  labs(
    x = "Heteroscedasticity parameter (c)",
    y = "Estimate (point ± 1.96 × MC-SE)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    panel.border = element_rect(color = "lightgray", fill = NA, linewidth = 0.5),
    panel.background = element_rect(fill = "white", color = NA),     # inside each panel
    plot.background  = element_rect(fill = "gray95", color = NA),
    strip.text = element_text(face = "bold", size = 10),            # bold facet titles
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank()
  )

p

tbl2 <- kbl(
  fits_table,
  caption   = "Simulation results for OLS regression under heteroscedasticity with Monte Carlo standard
  booktabs  = TRUE,
  align     = "cc",
  col.names = c(
    "c",
    "Bias",
    "\\makecell{Model\\\\SE}",
    "\\makecell{Empirical\\\\SE}",
    "\\makecell{Relative\\\\error in\\\\model SE\\\\(\\%)}",
    "\\makecell{CI\\\\Coverage\\\\(\\%)}"
  ),
  escape = FALSE   # allow LaTeX code inside names
) %>%
  kable_styling(
    latex_options = c("HOLD_position"),
    full_width    = FALSE,
    position      = "center",
    font_size     = 9
```

```
  )
tbl2
```